Unidad 6:

BASES DE DATOS DISTRIBUIDAS Y ALTA DISPONIBILIDAD

FICHA TÉCNICA DE LA UNIDAD

Unidad Didáctica	UD 6: Bases de Datos Distribuidas y Alta Disponibilidad
Resultados de Aprendizaje	**RA6:** Aplica criterios de integridad, disponibilidad y rendimiento en la administración de bases de datos distribuidas, analizando y configurando mecanismos de replicación y fragmentación.
Criterios de Evaluación	**CE 6.a:** Se han identificado las ventajas e inconvenientes de las bases de datos distribuidas. **CE 6.b:** Se han configurado y verificado escenarios de replicación. **CE 6.c:** Se han resuelto problemas de replicación. **CE 6.d:** Se han configurado y verificado escenarios de fragmentación de datos (sharding). **CE 6.e:** Se han analizado las características de los clústeres de bases de datos. **CE 6.f:** Se han utilizado herramientas de balanceo de carga. **CE 6.g:** Se han realizado pruebas de recuperación ante fallos.
Tiempo	9 horas lectivas (equivalente a unas 4 semanas de clase en el módulo ASGBD)

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

2. FUNDAMENTOS DE LAS BASES DE DATOS DISTRIBUIDAS (BDD)

    2.1. Definición y Conceptos Clave

    2.2. Arquitecturas Homogéneas vs. Heterogéneas

    2.3. Ventajas y Desventajas desde la Perspectiva del SysAdmin

3. MECANISMOS DE REPLICACIÓN EN MYSQL/MARIADB

    3.1. Replicación Asíncrona Tradicional (Source-Replica)

3.2. Replicación Semi-síncrona

3.3. Replicación Síncrona Virtual: MySQL Group Replication

3.4. El Binlog, Relay Log y el Rol Crítico de los GTID (Global Transaction Identifiers)

4. ALTA DISPONIBILIDAD Y CLÚSTERES DE BASES DE DATOS

4.1. Arquitectura MySQL InnoDB Cluster

4.2. Arquitectura MySQL NDB Cluster

5. FRAGMENTACIÓN Y SHARDING (PARTICIONADO DE DATOS)

5.1. Fragmentación Horizontal (Sharding) y Fragmentación Vertical

5.2. Orquestación y Balanceo con ProxySQL

6. CASOS PRÁCTICOS DE CONFIGURACIÓN Y SCRIPTS DE ADMINISTRACIÓN (SYSADMIN)

6.1. Script Bash para Monitorización Automática del Estado de la Réplica

6.2. Configuración GTID en mysqld.cnf para Alta Disponibilidad

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

En el entorno tecnológico actual, la disponibilidad de la información de forma ininterrumpida (24x7x365) es un pilar fundamental para cualquier organización. Un administrador de sistemas no puede permitir que un fallo de hardware aislado, un corte de red o la saturación de un servidor central detengan el acceso a los datos de la empresa.

La presente unidad aborda las técnicas y herramientas de diseño que permiten romper el modelo tradicional de almacenamiento centralizado (un único servidor base de datos) para dar paso a sistemas distribuidos, replicados y tolerantes a fallos.

Este contenido se conecta de manera directa y estratégica con otros módulos clave del ciclo formativo de grado superior de ASIR:

- **Implantación de Sistemas Operativos (ISO) / Administración de Sistemas Operativos (ASO):** Para comprender y gestionar los servicios del sistema, las rutas de logs de Linux, los niveles de permisos y el uso del planificador de tareas (cron).
- **Planificación y Administración de Redes (PAR) / Servicios de Red e Internet (SRI):** Dado que la comunicación entre nodos requiere un sólido conocimiento de direccionamiento IP, enrutamiento, puertos TCP, subredes, configuración de cortafuegos (iptables/ufw) y balanceadores de carga TCP/HTTP.

- **Seguridad y Alta Disponibilidad (SAD):** Donde se profundiza en las políticas de continuidad de negocio, métricas de tolerancia a fallos (RTO/RPO), sistemas RAID y el diseño de arquitecturas con redundancia activa.

2. FUNDAMENTOS DE LAS BASES DE DATOS DISTRIBUIDAS (BDD)

2.1. Definición y Conceptos Clave

Una **Base de Datos Distribuida (BDD)** es una colección de múltiples bases de datos que están interconectadas lógicamente y distribuidas físicamente a lo largo de una red de comunicaciones. Para el usuario o la aplicación cliente, el sistema debe comportarse como si fuera una base de datos centralizada única. Esta ilusión se conoce como **transparencia de distribución**.

A nivel de arquitectura de red, un sistema de base de datos distribuido se compone de un conjunto de ordenadores independientes (llamados **sitios** o **nodos**). Cada nodo tiene capacidad de procesamiento local y ejecuta su propio Sistema Gestor de Bases de Datos (SGBD).

2.2. Arquitecturas Homogéneas vs. Heterogéneas

Atendiendo al software que compone los nodos, las BDD se clasifican en:

- **Bases de Datos Distribuidas Homogéneas:**
  - Todos los nodos utilizan el mismo SGBD (por ejemplo, todas las instancias son MySQL 8.0).
  - Son conscientes de la existencia de los demás sitios y colaboran activamente para procesar las peticiones globales de forma transparente.
  - Son mucho más fáciles de diseñar, administrar y mantener. Es el estándar de oro en arquitecturas corporativas controladas por el mismo equipo de infraestructura.
- **Bases de Datos Distribuidas Heterogéneas:**
  - Diferentes nodos pueden usar SGBDs distintos (por ejemplo, algunos nodos con Oracle, otros con PostgreSQL y otros con MySQL).
  - El gran reto técnico aquí es la traducción de esquemas, dialectos SQL e interconectividad. Requieren capas intermedias de traducción de software (*middleware*) y no garantizan siempre la semántica transaccional estricta.

2.3. Ventajas y Desventajas desde la Perspectiva del SysAdmin

Como futuros administradores de sistemas (SysAdmins), toda decisión de diseño arquitectónico tiene asociadas ventajas técnicas y peajes operativos:

Ventajas

- **Rendimiento y Escalabilidad:** Al distribuir la carga de trabajo entre múltiples servidores, se reduce el cuello de botella en CPU e I/O de disco. Los datos de acceso frecuente se pueden ubicar físicamente en el nodo más cercano a los usuarios finales (localización de datos).
- **Alta Disponibilidad y Tolerancia a Fallos:** Si un nodo hardware falla, el sistema continúa funcionando a través de los nodos restantes que mantienen copias de los datos, evitando el punto único de fallo (*Single Point of Failure* o *SPOF*).
- **Eficiencia Económica:** En la actualidad, es económicamente mucho más viable y flexible escalar horizontalmente (desplegar múltiples servidores medianos/baratos virtualizados o en contenedores Docker) que escalar verticalmente (comprar una única máquina extremadamente potente y costosa).
- **Crecimiento Modular:** Facilita la adición de nuevos nodos para asumir mayor carga sin detener el servicio actual.

Desventajas

- **Gran Complejidad de Administración:** La configuración de red, la monitorización, la gestión de la consistencia global y la resolución de problemas de sincronización de datos son exponencialmente más complejas.
- **Problemas de Integridad de Datos (Teorema CAP):** Mantener la integridad referencial y las propiedades ACID globales (Atomicidad, Consistencia, Aislamiento y Durabilidad) en entornos distribuidos es muy costoso a nivel de latencia de red.
- **Seguridad Elevada:** Al transmitir información crítica a través de la red corporativa o internet entre nodos, se deben implementar obligatoriamente canales cifrados (TLS/SSL), VPNs internas y políticas estrictas de cortafuegos en cada nodo.
- **Sobrecarga de Red:** La cantidad de datos adicionales de control que se transmiten para mantener la consistencia e integridad referencial incrementa el consumo de ancho de banda.

3. MECANISMOS DE REPLICACIÓN EN MYSQL/MARIADB

La replicación es la técnica más común y efectiva para implementar alta disponibilidad a nivel de base de datos. Consiste en la copia automática y continua de las transacciones ejecutadas en un servidor de bases de datos de origen (denominado *Source* o *Maestro*) hacia uno o más servidores de destino (denominados *Replicas* o *Esclavos*).

+-----------------+ +-----------------+

| NODO SOURCE | --(Binlog Sync)--> | NODO REPLICA |

| (Lectura/Escrit) | | (Solo Lectura) |

+-----------------+ +-----------------+

3.1. Replicación Asíncrona Tradicional (Source-Replica)

Es el modelo de replicación por defecto en MySQL desde hace décadas. Su funcionamiento secuencial es el siguiente:

1. El cliente realiza una escritura (INSERT, UPDATE, DELETE) en el servidor *Source*.
2. El *Source* ejecuta localmente la consulta, la escribe en su registro binario (**Binary Log**) y confirma inmediatamente el éxito de la transacción al cliente.
3. De forma independiente (asíncrona), un hilo de red en la máquina *Replica* solicita los nuevos registros del log del *Source*, los descarga a su propio disco en un archivo llamado **Relay Log** e inicia la replicación en su base de datos local.

- **Ventaja:** Máximo rendimiento de escritura. El nodo *Source* no espera a que los cambios se transmitan por la red para confirmar la transacción al cliente.
- **Inconveniente:** Riesgo inherente de pérdida de datos en caso de desastre. Si el *Source* sufre un corte de energía letal antes de que el log se transmita al nodo *Replica*, al promocionar el nodo *Replica* a nuevo *Source* se perderán aquellas transacciones aún no transmitidas.

3.2. Replicación Semi-síncrona

Introduce un mecanismo de control de seguridad intermedio:

1. El cliente envía la escritura al *Source*.
2. El *Source* registra la transacción en su *Binary Log* local y la envía a la red para las *Replicas*.
3. El *Source* **bloquea la respuesta** al cliente hasta recibir la confirmación (ACK) de que al menos **un** nodo de réplica ha recibido los eventos de transacción y los ha guardado en su *Relay Log*.
4. Una vez recibido el ACK de red, el *Source* responde con éxito al cliente.

- **Ventaja:** Garantiza que no habrá pérdida de transacciones confirmadas si el *Source* falla, ya que al menos una réplica tendrá una copia de los registros de log en su disco.
- **Inconveniente:** Mayor latencia en las escrituras debido a los tiempos de ida y vuelta de red (*RTT*) necesarios para recibir los ACKs.

3.3. Replicación Síncrona Virtual: MySQL Group Replication

Se basa en un modelo de compartición total del estado (*state machine replication*):

- El clúster está compuesto por un grupo de servidores (habitualmente un mínimo de 3 para garantizar quórum).
- Se utiliza un protocolo de consenso mutuo de red basado en el algoritmo **Paxos** (o la implementación interna de MySQL llamada *Group Communication System*).
- Cuando una transacción se procesa en cualquier nodo, este emite una propuesta al grupo de red. La transacción no se confirma localmente en ningún nodo hasta que la **mayoría absoluta del grupo** (quórum) vote favorablemente que no existen conflictos de integridad y se acuerde el orden exacto de aplicación.
- Ofrece dos modos de operación:
  - **Single-Primary:** Solo un nodo acepta lecturas y escrituras; los demás son de solo lectura de manera automática. Si el primario falla, el clúster elige automáticamente un nuevo primario mediante votación de consenso.
  - **Multi-Primary:** Todos los nodos aceptan escrituras concurrentes. Extremadamente útil pero requiere una cuidada gestión de la lógica de negocio para evitar colisiones de claves primarias.

3.4. El Binlog, Relay Log y el Rol Crítico de los GTID (Global Transaction Identifiers)

Para auditar y depurar problemas en escenarios de replicación, un SysAdmin debe dominar los componentes internos que la hacen posible:

- **Binary Log (binlog):** Fichero donde el servidor *Source* anota todas las modificaciones realizadas a nivel de base de datos (tanto DDL como DML). Es la fuente de la verdad para el clúster.
- **Relay Log:** Fichero local en el servidor *Replica* donde se guardan temporalmente las transacciones descargadas del *Source*, antes de que el hilo de ejecución SQL las procese en la base de datos de la réplica.
- **GTID (Global Transaction Identifier):** Identificador global único que se asocia a cada transacción que se confirma en el servidor de origen. Se compone de un UUID identificativo del nodo emisor y un número de secuencia secuencial (ej: 3E11FA47-71EF-11E1-9E3C-C80AA9429562:1045).
  - **¿Por qué son una revolución frente a la replicación tradicional por posición de log binario (binlog_file y binlog_pos)?** * En la replicación tradicional, si el maestro fallaba, para apuntar las réplicas a

un nuevo maestro el SysAdmin debía buscar manualmente el archivo y la posición exacta de bytes donde se detuvo cada réplica.

- Con **GTID**, el seguimiento es automático: las réplicas le preguntan al nuevo maestro qué transacciones con GTID les faltan y el maestro simplemente les envía el diferencial, evitando errores humanos catastróficos en el proceso de failover.

4. ALTA DISPONIBILIDAD Y CLÚSTERES DE BASES DE DATOS

4.1. Arquitectura MySQL InnoDB Cluster

Es la solución de alta disponibilidad llave en mano proporcionada por Oracle para aplicaciones tradicionales. Integra de forma nativa tres tecnologías:

1. **MySQL Group Replication:** El motor encargado de replicar síncronamente los datos entre los servidores y gestionar la resiliencia y el quórum del clúster de base de datos.
2. **MySQL Shell:** El cliente avanzado y entorno de administración que expone la API AdminAPI. Permite configurar, validar y arrancar un clúster de alta disponibilidad completo con un par de comandos JS/Python.
3. **MySQL Router:** Un middleware ligero que actúa como proxy de red o balanceador de carga entre las aplicaciones cliente y los servidores de base de datos. Expone puertos separados de lectura y escritura (por ejemplo, el puerto 6446 para escrituras redirigido al Primary, y el puerto 6447 para lecturas balanceadas en Round-Robin entre las Replicas).

4.2. Arquitectura MySQL NDB Cluster

Es una arquitectura de clúster distribuido nativo sin almacenamiento compartido (arquitectura *shared-nothing*). Se diseñó originalmente para el sector de las telecomunicaciones debido a su bajísima latencia de respuesta y disponibilidad extrema.

La arquitectura física divide las tareas en tres tipos de roles o nodos independientes:

- **Nodos de Administración (Management Nodes - ndb_mgmd):** Encargados de almacenar los datos de configuración del clúster, coordinar los arranques iniciales de los nodos, controlar el quórum y gestionar las copias de seguridad globales.
- **Nodos de Datos (Data Nodes - ndbd o ndbmtd):** Alojan la base de datos en memoria RAM o disco. El motor de almacenamiento utilizado es obligatoriamente **NDBCLUSTER**. Los datos se fragmentan automáticamente y se replican entre grupos de nodos de datos.
- **Nodos SQL / Servidores MySQL (SQL Nodes - mysqld):** Actúan como la interfaz de acceso de cara a las aplicaciones externas. Reciben las

sentencias SQL estándar de los clientes, las traducen al protocolo interno de red NDB y las envían a los Data Nodes para su procesamiento.

5. FRAGMENTACIÓN Y SHARDING (PARTICIONADO DE DATOS)

Cuando el volumen de escrituras de nuestra aplicación de producción supera la capacidad máxima de procesamiento de un único servidor, la replicación clásica de alta disponibilidad ya no es suficiente, pues todas las escrituras se siguen embotellando en el único nodo primario. Aquí es donde entra en juego el particionado distribuido.

5.1. Fragmentación Horizontal (Sharding) y Fragmentación Vertical

- **Fragmentación Vertical:** Consiste en separar columnas de una tabla en bases de datos o nodos distintos. Por ejemplo, almacenar columnas de uso frecuente (como ID_Usuario y Login) en un nodo rápido de disco de estado sólido, y las columnas pesadas (como Foto_Perfil_Blob o Descripcion_Historial) en un nodo de almacenamiento secundario lento.
- **Fragmentación Horizontal o Sharding:** Consiste en dividir las filas de una tabla y repartirlas entre diferentes nodos físicos independientes (llamados **Shards**). Por ejemplo, un rango de ID de clientes del 1 al 1.000.000 se almacena en el Shard-A y el rango del 1.000.01 al 2.000.000 en el Shard-B.

5.2. Orquestación y Balanceo con ProxySQL

La gran desventaja teórica del Sharding es que la aplicación cliente no sabe de forma nativa en qué nodo está cada registro, por lo que tendría que gestionar complejas conexiones simultáneas.

Para resolver este desafío de manera transparente se utiliza **ProxySQL**, un proxy de bases de datos de alto rendimiento y bajo consumo de CPU:

- ProxySQL actúa como intermediario entre la aplicación y los Shards.
- Almacena reglas de enrutamiento basadas en expresiones regulares (*regex*).
- `Cuando recibe una consulta (ej: SELECT * FROM clientes WHERE id = 45;), analiza el texto de la consulta, detecta el ID utilizando patrones regex, comprueba su tabla de reglas de enrutamiento y redirige silenciosamente la consulta al Shard correspondiente, devolviendo el resultado de forma transparente al usuario.`

6. CASOS PRÁCTICOS DE CONFIGURACIÓN Y SCRIPTS DE ADMINISTRACIÓN (SYSADMIN)

6.1. Script Bash para Monitorización Automática del Estado de la Réplica

Como administradores de sistemas, no podemos vigilar manualmente las consolas de base de datos a diario. Debemos automatizar alarmas que nos alerten inmediatamente de desincronizaciones en la réplica de MySQL.

```
#!/bin/bash
#
================================================================================
# SCRIPT DE MONITORIZACIÓN PARA MYSQL REPLICA - ASGBD (UD 6)
#
================================================================================
# Variables de entorno y credenciales
DB_USER="sysadmin_monitor"
DB_PASS="SecurePassword_2026!"
DB_HOST="localhost"
LOG_FILE="/var/log/mysql/replica_monitor.log"
# Comprobación de existencia del ejecutable mysql
if ! command -v mysql &gt; /dev/null; then
echo "$(date '+%Y-%m-%d %H:%M:%S') [CRITICAL] El cliente mysql no
está instalado." &gt;&gt; "$LOG_FILE"
exit 1
fi
# Extracción de parámetros críticos de la réplica utilizando AWK
REPLICA_STATUS=$(mysql -u "$DB_USER" -p"$DB_PASS" -h "$DB_HOST" -e
"SHOW REPLICA STATUS\G" 2&gt;/dev/null)
if [ -z "$REPLICA_STATUS" ]; then
echo "$(date '+%Y-%m-%d %H:%M:%S') [ERROR] No se pudo obtener el
estado de la réplica. ¿MySQL está levantado?" &gt;&gt;
"$LOG_FILE"
exit 1
fi
# Parsers de estados de hilos
IO_RUNNING=$(echo "$REPLICA_STATUS" | grep -i "Replica_IO_Running:" |
awk '{print $2}')
SQL_RUNNING=$(echo "$REPLICA_STATUS" | grep -i "Replica_SQL_Running:"
| awk '{print $2}')
```

```
SECONDS_BEHIND=$(echo "$REPLICA_STATUS" | grep -i
"Seconds_Behind_Source:" | awk '{print $2}')
# Imprimir variables para depuración (opcional)
# echo "IO: $IO_RUNNING | SQL: $SQL_RUNNING | Lag:
$SECONDS_BEHIND"
# Evaluación de la salud del clúster
if [ "$IO_RUNNING" != "Yes" ] || [ "$SQL_RUNNING" != "Yes" ];
then
echo "$(date '+%Y-%m-%d %H:%M:%S') [ALERT] ¡Réplica caída!
IO_Running=$IO_RUNNING, SQL_Running=$SQL_RUNNING" &gt;&gt;
"$LOG_FILE"
# Aquí un SysAdmin real añadiría un envío de correo por ssmtp,
telegram o webhook de Slack
# mail -s "ALERTA: Réplica Caída en Host $(hostname)"
alert@empresa.com &lt;&lt;&lt; "Detalle:
$REPLICA_STATUS"
else
# Si todo está OK, verificamos si hay lag de tiempo sustancial
if [ "$SECONDS_BEHIND" != "NULL" ] &amp;&amp; [
"$SECONDS_BEHIND" -gt 300 ]; then
echo "$(date '+%Y-%m-%d %H:%M:%S') [WARNING] Retraso de réplica
elevado: $SECONDS_BEHIND segundos." &gt;&gt; "$LOG_FILE"
else
echo "$(date '+%Y-%m-%d %H:%M:%S') [OK] Réplica funcionando
correctamente sin retrasos." &gt;&gt; "$LOG_FILE"
fi
fi
```

Este script analiza la salida del comando SHOW REPLICA STATUS (o SHOW SLAVE STATUS en entornos compatibles) para validar el estado de los dos hilos principales: **Replica_IO_Running** (hilo de red que descarga los logs del maestro) y **Replica_SQL_Running** (hilo que aplica las consultas en local).

6.2. Configuración GTID en mysqld.cnf para Alta Disponibilidad

Para habilitar de manera segura la replicación basada en GTID en nuestros servidores de base de datos MySQL, debemos configurar los parámetros adecuados en el fichero de configuración de MySQL (normalmente ubicado en /etc/mysql/mysql.conf.d/mysqld.cnf en sistemas basados en Debian/Ubuntu).

```
[mysqld]
# Identificador único de nodo (Entero entre 1 y 4294967295)
```

```
# Debe ser diferente en cada uno de los servidores del clúster
server-id = 101
# Habilitar el registro binario y ruta absoluta de almacenamiento de
logs
log_bin = /var/log/mysql/mysql-bin.log
# El formato ROW es obligatorio para garantizar la consistencia en
replicación GTID
binlog_format = ROW
# Activar de forma obligatoria el modo de Transacciones por ID Global
(GTID)
gtid_mode = ON
# Evitar la confirmación de transacciones que rompan la consistencia
GTID
# Impide la ejecución de sentencias no seguras (como CREATE TABLE ...
SELECT)
enforce_gtid_consistency = ON
# Forzar a la réplica a escribir los cambios importados también en su
binlog local
# Esencial para clústeres multi-maestro y esquemas de
encadenamiento
log_replica_updates = ON
# Forzar consistencia de escrituras en disco ante caídas de energía
eléctrica (batería RAID)
sync_binlog = 1
innodb_flush_log_at_trx_commit = 1
```

La siguiente plantilla de configuración es apta para producción e indispensable para que funcione la tecnología GTID de manera limpia y sin corrupciones de datos: